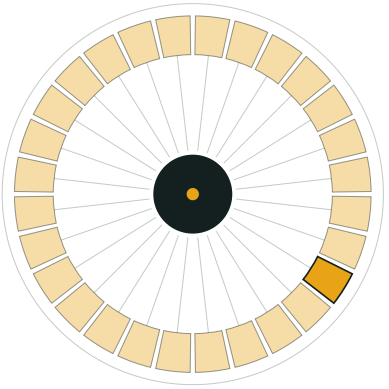
Gestionar el error en la era de la IA

del panóptico a la cultura del aprendizaje



Autor Martín Dopazo

Versión Borrador v3 — 28 de octubre de 2026

Resumen

Con la IA escribimos código mucho más rápido de lo que podemos revisarlo, entenderlo y hacernos cargo de él. El análisis que antes pasaba antes de producción (caminos posibles, casos borde, reglas de negocio) se muda a las aplicaciones ya en uso. Lo que planteo en este trabajo es que la respuesta de la gestión no puede ser más control, sino un entorno donde el error se vea, se discuta y se valore como fuente de conocimiento. Repaso cómo las herramientas de monitoreo que usamos todos los días empujan a esconder los errores, propongo distinguir culpa de responsabilidad y dejo tres líneas de acción con dos herramientas concretas: una plantilla de post-mortem sin culpa (Anexo A) y unas pautas para dailys sin vigilancia (Anexo B).

1 El problema: el código viaja más rápido que la comprensión

El código se produce a una velocidad mayor que la que permite evaluarlo, analizarlo y revisarlo, y eso pasa en cada instancia de control:

- **En el debug del propio desarrollador**, que acepta lo que "parece funcionar".
- **En los tests**, que las IAs tienden a orientar al camino feliz.
- **En los Pull Requests**, que muchas veces no se revisan a fondo: se los lee en diagonal y se verifica que compile.

El resultado es que el momento de análisis (caminos posibles, situaciones borde, reglas de negocio) se traslada a producción. Los problemas aparecen con el sistema ya en uso, frente a los usuarios.

Ariel Fabian (2026) bautizó este fenómeno **"efecto bullet"**, por analogía con el ajedrez ultrarrápido, donde el poco tiempo anula el cálculo profundo. Su tesis es que el problema no es la herramienta sino el ritmo: una persona necesita tiempo para entender la lógica de un software y no puede hacerlo a la velocidad con la que un modelo genera texto. Cuando eso pasa, se acumula **deuda cognitiva**: sistemas que funcionan pero que nadie en el equipo sabe modificar con seguridad, y cuyas fallas pueden llegar directo a producción. Fabian apoya el diagnóstico en evidencia reciente que él cita:

- En 196.000 Pull Requests, la revisión humana profunda cayó del 48 % al 21 % (He y Vasilescu, 2026).
- En organizaciones con adopción masiva de IA y sin reglas, los cambios aprobados sin revisión previa subieron un 76,3 % y los incidentes críticos en producción un 125,4 % (Faros AI, 2026; es un patrón observacional).
- En un ensayo controlado con desarrolladores, quienes delegaron en la IA comprendieron 17 puntos porcentuales menos de los conceptos (Shen y Tamkin, 2025).

Fabian propone cinco reglas de trabajo adaptadas del ajedrez (control de ritmo, la regla del 10 + 10, un "peaje cognitivo" antes del commit, auditoría preventiva y el modelo centauro). Y reconoce una limitación que para mí es el punto de partida de este trabajo: *ninguna regla técnica funciona de forma aislada si los indicadores de seguimiento solo miran el volumen de entrega inmediato*; cuando el apuro del calendario le gana al espacio de análisis, cualquier control se convierte en un paso formal que se completa por cumplir.

Yo parto de ese diagnóstico y me concentro en **la otra mitad del problema**: qué le pasa a un equipo cuando, además de la presión de velocidad, trabaja en un entorno donde equivocarse tiene consecuencias. Las reglas de ritmo y asimilación (el peaje cognitivo, el 10 + 10) solo funcionan si el equipo puede decir "esto no lo entiendo" sin pagar un costo, y si lo que se mide no premia la velocidad aparente. Eso no lo garantiza ninguna regla técnica: es una decisión de gestión.

2 Hipótesis

> Los equipos que hacen visible, analizable y valorado el error comprenden mejor lo que entregan, y esa comprensión es la mejor defensa que tenemos contra la deuda cognitiva del código generado con IA.

De acá sale un desafío para quienes gestionamos equipos: generar un entorno de **revisión, replanteo, cooperación y transparencia** (*disclosure*) sobre lo que sabemos y lo que no sabemos del modelo, la arquitectura y las funcionalidades que estamos desarrollando.

Y, a la vez, evitar la deuda cognitiva de usar código que no se entiende o no se revisó. Su consecuencia es la **ausencia de propósito arquitectónico**: equipos que desarrollan y dan soporte a sistemas construidos a toda velocidad, que nadie comprende y de los que nadie es dueño.

3 Por qué los equipos esconden el error

3.1 Estilos de conducción

Hay varios estilos de conducir un equipo de desarrollo. Algunos buscan al responsable del error. Otros le restan importancia, pero igual miden la "productividad" de cada desarrollador por los errores que comete.

Y hay una tercera actitud que parece sana pero también hay que mirar con desconfianza: *"no importa quién se equivocó, no busco responsables sino soluciones"*. Sin querer, esa postura **vela el error en función de la solución**: se arregla, pero no se aprende nada sobre el desarrollo ni sobre el porqué del error. El problema se resuelve y el aprendizaje se pierde.

3.2 El panóptico como metáfora de gestión

El exceso de control (reuniones permanentes donde hay que dar cuenta de avances y demoras) logra lo contrario de lo que busca: se *esconden* los problemas, o se escribe código que "salva" lo que se ve sin que nadie sepa con precisión qué hace ni qué otros problemas puede traer.

Michel Foucault, en *Vigilar y castigar*, describe el panóptico como un dispositivo que se aplica a escuelas, hospitales, fábricas y cuarteles para ordenar, corregir y normalizar conductas. Lo que lo hace eficaz es que el control se **internaliza**: como nunca sabemos con certeza si nos están mirando en este momento, terminamos actuando como si siempre nos miraran.

Byung-Chul Han (*Psicopolítica*, *La sociedad del cansancio*) agrega que el poder hoy ya no funciona tanto por disciplina externa como por exposición voluntaria y autoexplotación: el panóptico digital es más sutil y más eficaz porque no necesita un vigilante. Esa diferencia importa acá. En un equipo de desarrollo la presión casi nunca viene de un jefe que mira, sino de un tablero que se actualiza y de una disponibilidad que se exhibe.

> **Una aclaración sobre la metáfora.** No digo que Jira, las dailys o el pair programming sean vigilancia en sí mismos. Digo que, **cuando se usan para controlar a la persona y no para coordinar el trabajo**, producen los efectos que describo acá. Vuelvo sobre esto en la sección 8.

3.3 Las herramientas como parte del dispositivo

Herramienta o práctica	Propósito declarado	Efecto real posible
Aplicaciones en segundo plano (Teams, Discord)	Facilitar la comunicación inmediata y resolver bloqueos	Monitoreo del tiempo de inactividad
Canales de voz permanentes	Simular la cercanía de la oficina física en remoto	Pérdida de privacidad y de espacio de concentración
Métricas de repositorio (commits, líneas, PRs)	Medir la estabilidad y el avance del proyecto	Cuantificación del esfuerzo humano como si fuera rendimiento

Cómo se ve esto en la práctica:

- **Presencia constante.** Los estados ("Disponible", "Ausente", "En una reunión") funcionan como las ventanas iluminadas de las celdas de Bentham. Sabemos que nuestra disponibilidad se ve en tiempo real, y eso genera la presión de estar "siempre activo".

- **Reuniones y voz permanente.** Las dailys que se alargan, el pair programming obligatorio y los canales de voz abiertos todo el día se justifican como "colaboración orgánica", pero dejan expuesto el tiempo y el ritmo de pensamiento de quien programa y, con eso, sus posibles errores.
- **El panóptico en el código.** El historial de commits (frecuencia, líneas, hora de entrega) y las métricas de gestión (tiempo por tarea, burndown) terminan funcionando como registros de rendimiento. Importa más la cantidad de commits o PRs que la decisión arquitectónica de organizarlos para que se entiendan y se puedan revisar.

El agravante de la IA. La generación asistida infla justamente las métricas de actividad: más commits, más líneas, más PRs en menos tiempo. Si usamos esas métricas para evaluar, cada vez medimos más ruido y menos comprensión. Y las métricas de *uso* de la IA (aceptación de sugerencias, volumen generado) empiezan a ser un nuevo tablero de control.

Esto ya está pasando. Según reportó la prensa especializada, Meta llegó a incluir el consumo de tokens de IA en las evaluaciones de sus ingenieros: había tablas de clasificación y la gente hacía tareas adicionales solo para mejorar su puntaje, lo que se conoció como *tokenmaxxing*. En septiembre de 2026, según un memo interno, la empresa resolvió no usar más los paneles de adopción de IA ni el conteo de tokens para evaluar impacto, y volver a mirar la calidad del resultado, la velocidad y la complejidad de los problemas (Cooter, 2026). Su propio CTO ya había advertido que el uso de tokens, por sí solo, no mide impacto de ninguna clase. Es la ley de Goodhart en vivo: cuando una medida se vuelve objetivo, deja de ser una buena medida. Esa misma ley la citaba un gerente de una fintech en un relevamiento sobre los paneles de uso de IA, que también registraba la desconfianza de los desarrolladores frente a datos tan detallados sobre cómo usan sus herramientas (Lazzaro, 2025).

3.4 La consecuencia: la autocensura

Lo más profundo no es el castigo posible sino la **autocensura**. En el desarrollo de software se ve en comportamientos muy concretos:

- **Teatralidad de la productividad.** Se priorizan las tareas visibles (mover tarjetas, escribir en canales públicos) sobre el trabajo profundo (*deep work*) y los problemas complejos, que necesitan tiempo de desconexión.
- **Ansiedad de disponibilidad.** El miedo a aparecer como "Ausente" fragmenta la atención: cortamos el flujo para contestar alertas y sacrificamos calidad de código para demostrar presencia.
- **Ilusión de autonomía.** Las metodologías ágiles se presentan como marcos de autoorganización, pero si dependen de un monitoreo estricto de datos y de reuniones constantes, la autonomía se vuelve **autovigilancia coordinada**: el equipo se disciplina solo para cumplir con la métrica del tablero.

Programar requiere abstracción y, sobre todo, entender el problema. El panóptico digital pide otra cosa: visibilidad y ruido comunicativo. El resultado es código que hoy funciona y que mañana no sabemos explicar, más un desgaste (burnout) que se paga en las personas.

4 El error como objeto de conocimiento

Para armar un entorno amigable con el error y con lo que no sabemos, me apoyo en las actitudes de los ambientes de aprendizaje. En las teorías del aprendizaje el error es parte del proceso de construir conocimiento. No es necesariamente una falla ni un fracaso: es, muy probablemente, **evidencia de una hipótesis en crisis**, la lógica que está detrás del error.

La estrategia de fondo es **aceptar, valorar y discutir** el error: no justificarse ni defenderse, sino explicarlo. Y no solo lo funcional, sino **la lógica que nos llevó a producirlo**.

Un error puede tener muchos orígenes:

- un prompt mal formulado;
- un mal análisis de la regla de negocio que el software tiene que cumplir;
- situaciones borde que no se contemplaron;
- entornos con condiciones de excepción (sin conectividad, caída de la base de datos, etc.);
- una infraestructura que no soporta los requerimientos.

Solo si valoramos y aceptamos el error llegamos a entender el modelo y los flujos del software en acción. Y eso, además, nos ayuda a anticipar problemas con más precisión y a diagnosticar mejor los que vayan apareciendo.

5 Responsabilidad sin culpa

La cultura sin culpa recibe una objeción previsible: si nadie es culpable, ¿quién se hace cargo? Mi respuesta es que **culpa y responsabilidad son cosas distintas**, y que una cultura del error necesita la segunda justamente para poder prescindir de la primera.

	Culpa	Responsabilidad
Mira hacia	Atrás: el daño ya causado	Adelante: la reparación y la prevención
Pregunta	¿Quién lo hizo? ¿Qué merece?	¿Qué pasó? ¿Quién necesita qué? ¿Qué cambiamos?
Efecto en el equipo	Se oculta información	Se comparte información
Resultado	Castigo	Aprendizaje y mejora del sistema

La distinción tiene antecedentes. Sidney Dekker, en su trabajo sobre *cultura justa* (*Just Culture: Balancing Safety and Accountability*), contrapone una versión **retributiva**, que clasifica las conductas y pregunta qué castigo corresponde, con una versión **restaurativa**, que pregunta quién resultó afectado, qué necesita y a quién le toca atenderlo. John Allspaw, cuando introdujo los post-mortems sin culpa en Etsy (2012), lo planteó en términos de gestión: mirar los errores “con una perspectiva de aprendizaje” y equilibrar seguridad y responsabilidad, no eliminar esta última.

Aplicado al desarrollo con IA, esto se traduce en una regla concreta:

> **Quien aprueba se hace responsable.**
>
> Tiene que poder explicar qué hace el cambio, por qué lo hace y qué casos no cubre, aunque no lo haya escrito. “Fue un error de la IA” no sirve como explicación, ni para quien desarrolla ni para quien gestiona.

La regla no castiga el error: **señala la opacidad**. Si alguien no puede explicar un cambio, lo esperable no es una sanción sino una reparación: leerlo entre varios, escribir la prueba que falta o deshacer el cambio. Lo que se cuida es la comprensión del equipo, no la reputación de nadie.

Esto no quiere decir que desconozca que existen conductas que ninguna cultura sana ampara, como la negligencia deliberada; la dejo fuera de este trabajo. Lo que sí propongo es una precaución antes de señalar a alguien: preguntarse si el sistema premia esa conducta.

> Quien aprueba un PR sin leerlo en un entorno donde se miden los PRs cerrados por semana no está violando las reglas: las está cumpliendo.

Por último, hay una irresponsabilidad que esta cultura sí quiere eliminar: **ocultar el error**. Quien lo oculta no se hace cargo ni de la falla ni de lo que el equipo podría aprender de ella. Por eso la seguridad psicológica no es lo opuesto de la responsabilidad: es lo que la hace posible.

6 Propuesta: tres líneas de acción

La gestión de los equipos tiene que pasar de una cultura del **monitoreo de presencia** a una cultura del **impacto y la seguridad psicológica**. En software, el error no es una falla de la persona: es un subproducto inevitable de explorar, de intentar resolver problemas y de innovar. Con estas estrategias, la gestión deja de ser la torre central de vigilancia y se convierte en **facilitadora de infraestructura y confianza**, para que desarrollar software vuelva a ser una disciplina creativa e iterativa.

6.1 Reconfigurar la comunicación y la presencia

- **Soberanía sobre el estado de disponibilidad.** Una política explícita de “sin indicadores”: que sea normal cerrar la mensajería o ponerse en “No molestar” en bloques de 3 o 4 horas para el trabajo profundo.
- **Comunicación asincrónica por defecto.** Una respuesta puede tardar 1 o 2 horas sin que eso signifique falta de compromiso. Para lo urgente, se define un canal de escalamiento específico (por ejemplo, un llamado).

- **Sin salas de voz permanentes.** Reemplazarlas por "horas de oficina" acotadas (30 minutos al inicio o al final del día, por ejemplo) o por salas de diseño puntuales. Colaborar tiene que ser una decisión consciente, no una exposición continua.

6.2 Redefinir las métricas

- **Impacto, no actividad.** Evitar las líneas de código, las horas conectado o la cantidad de commits como medida de rendimiento. Medir el valor entregado al usuario, la estabilidad del sistema, el cumplimiento de los objetivos del sprint y, sobre todo, **la comprensión técnica de lo que se aporta al sistema**.
- **El tablero como mapa, no como látigo.** Las herramientas de gestión sirven para que el equipo coordine dependencias, no para que se microgestione cada tarea. Si una tarjeta se atrasa, la pregunta es "*¿cómo te ayudo a destrabarla?*" y no "*¿por qué tardaste más?*". Y si la traba revela algo relevante del sistema, se analiza a fondo y, una vez resuelta, se comparte el hallazgo con el equipo.

6.3 Institucionalizar el error como parte natural del trabajo

Para que el equipo deje de ocultar fallas por miedo a la vigilancia, el error tiene que ser visible, analizado y entendido como fuente de mejora.

- **Post-mortems sin culpa.** Ante una falla grave en producción o un bug crítico, la reunión se concentra en cómo y por qué falló el sistema, no en quién se equivocó. Se buscan fallas de proceso, falta de pruebas automatizadas o ambigüedad en los requerimientos. En equipos que usan IA, el análisis incluye el **prompt, el contexto que se le dio y qué se revisó realmente** (plantilla en el Anexo A).
- **Entornos de experimentación seguros.** Ambientes locales o de staging sólidos donde romper el código sea barato y seguro. Si sé que un error en mi rama no tiene consecuencias punitivas ni exposición negativa, me animo a buscar mejores soluciones de arquitectura.
- **Sesiones de aprendizaje compartido (FailFaire).** Espacios mensuales donde líderes técnicos y desarrolladores senior cuentan sus errores recientes y qué aprendieron. Ver vulnerables a los referentes desarma la ilusión panóptica de la perfección obligatoria.
- **Dailys sin vigilancia.** La daily deja de ser un reporte individual y pasa a ser una inspección del trabajo del equipo (pautas en el Anexo B).

7 Cómo saber si funciona

Una propuesta sobre métricas tiene que poder medirse, cuidando de no reproducir el problema que critica. Algunos indicadores posibles:

- **Estabilidad del delivery:** tasa de fallos por cambio y tiempo de recuperación (métricas DORA).
- **Aprendizaje:** cantidad de post-mortems hechos y de acciones preventivas efectivamente cerradas.
- **Comprensión:** porcentaje de cambios en los que quien aprobó pudo explicarlos, relevado por muestreo y no persona por persona.

- **Clima:** una encuesta breve y anónima de seguridad psicológica (por ejemplo, la escala de Edmondson).

Medir la adopción de una cultura sin culpa con instrumentos individuales y permanentes sería crear un panóptico nuevo. Por eso los indicadores se relevan **a nivel de equipo, por muestreo y con un propósito declarado**.

8 Límites y objeciones

Una propuesta de gestión tiene que poder discutirse. Estas son las objeciones más fuertes que veo y cómo las respondo, o las dejo abiertas.

8.1 No todo control es vigilancia

Un equipo necesita coordinarse, y los equipos remotos más todavía. La comunicación asincrónica evita interrupciones, pero tiene costos: demoras, malentendidos, decisiones que quedan esperando una respuesta. Y hay momentos (un incidente en curso, un diseño en discusión) que piden una conversación en tiempo real. Por eso no propongo eliminar la sincronía sino **hacerla deliberada**: encuentros con propósito y duración acotados, y canales de escalamiento claros para lo urgente. A la vez, tener en cuenta la siguiente declaración: **Si todo es urgente, nada es urgente**.

8.2 Las herramientas no son el problema

Ni Jira, ni las dailys, ni el pair programming son vigilancia en sí mismos. El mismo tablero puede ser un mapa que ayuda a coordinar o un látigo que mide minutos. La metáfora del panóptico describe un **uso** de las herramientas (controlar a la persona), no las herramientas.

8.3 Registro para auditar no es lo mismo que registro para evaluar

Muchos entornos, sobre todo los regulados, **exigen trazabilidad**: quién cambió qué y cuándo. No propongo renunciar a ese registro, sino separar su finalidad: el historial sirve para auditar y para reconstruir un incidente, pero no para medir el desempeño de las personas ni para armar rankings de errores.

8.4 Trazabilidad como responsabilidad, no como culpabilidad

Rastrear qué cambio originó un incidente y quién lo aprobó es una práctica legítima y necesaria. Los diseños experimentales sobre desarrollo con IA, como el protocolo que propone Fabian (2026), lo contemplan al seguir el commit causante de una falla y registrar la autoría del código. Desde mi perspectiva, esa auditoría se entiende como **responsabilidad** (la capacidad de reconstruir qué pasó, entender por qué y reparar) y no como **culpabilidad** (*blame*). La diferencia no está en el dato sino en el uso: el mismo historial sirve para aprender o para señalar. Para que siga siendo lo primero, los indicadores se agregan a nivel de equipo, se informan antes de empezar y no se usan para evaluar personas.

8.5 Un equipo no cambia solo

Un equipo puede adoptar dailys sin vigilancia y post-mortems sin culpa, pero si la organización que lo rodea evalúa por tickets cerrados, líneas generadas o velocidad de entrega, la cultura sin culpa queda en puro barniz. Fabian (2026) advierte algo equivalente sobre sus reglas: ningún control funciona de forma aislada si los indicadores solo premian el volumen inmediato. Hace falta **coherencia entre niveles**: lo que dice el equipo, lo que mide la gestión y lo que recompensa la organización.

8.6 El riesgo del ritual vacío

Un post-mortem sin culpa que no produce acciones cerradas es otra forma de teatro. Lo mismo pasa con una daily "sin vigilancia" que sigue siendo un reporte individual con otro nombre. Por eso los anexos piden acciones con dueño y fecha, y una revisión periódica de su cumplimiento.

8.7 Alcance y evidencia

Esta propuesta se apoya en teoría (aprendizaje, cultura justa, seguridad psicológica) y en mi experiencia en la gestión de equipos de desarrollo. Tampoco existe una medida única de "cultura del error", y es probable que el contexto (tamaño del equipo, sector, nivel de regulación, madurez) cambie su efecto.

Como línea a futuro, el diseño factorial de Fabian (reserva de tiempo de asimilación y peaje cognitivo) podría sumar un tercer factor: el clima del equipo frente al error. La pregunta sería si las reglas de ritmo y comprensión funcionan mejor, igual o peor en equipos con alta seguridad psicológica. Es una hipótesis, no un resultado.

9 Conclusión

La IA no inventó el problema del error en los equipos de software, pero lo volvió más costoso. Cuando el código se produce más rápido de lo que podemos entenderlo, lo que no se revisó llega a producción y lo que nadie entiende no tiene dueño. Las reglas de ritmo y asimilación, como las de Fabian, atacan esa velocidad. Yo sostengo que esas reglas necesitan una condición previa: **un entorno donde decir "esto no lo entiendo" no cueste nada.**

Ese entorno no se consigue con más control. Las herramientas de monitoreo, las reuniones permanentes y las métricas de actividad, cuando se usan para vigilar a la persona y no para coordinar el trabajo, empujan a esconder los problemas y a producir código que funciona sin que nadie sepa por qué. La alternativa que propongo se apoya en tres ideas:

1. **El error es evidencia de una hipótesis en crisis**: un objeto de análisis y de aprendizaje, no un motivo de vergüenza.
2. **Responsabilidad no es culpa.** Quien aprueba se hace responsable de entender lo que entrega; el análisis de una falla se concentra en cómo el sistema la permitió.
3. **Lo que se mide y se recompensa tiene que ser coherente con lo que se dice.** Si no, la cultura sin culpa queda en una declaración sin efecto.

Las herramientas concretas (post-mortems sin culpa, dailys centradas en el tablero, métricas de impacto, espacios para compartir errores) son un punto de partida para probar en equipos reales y ajustar con datos. Todavía no tengo evidencia propia, y decirlo es parte de ser coherente: un modelo que pide valorar el error tiene que poder equivocarse en público.

La velocidad de la IA no va a frenar. Lo que sí depende de quienes gestionamos equipos es decidir qué pasa cuando alguien se equivoca: si se esconde o si se estudia. De esa decisión depende que el equipo siga entendiendo lo que construye.

Referencias

Fabian, A. (2026). *El Efecto Bullet: Deuda Cognitiva y la Ilusión de la Velocidad en la Era de la IA*. Zenodo. DOI 10.5281/zenodo.23641059. https://arfab.github.io/efectobullet/

Cooter, M. (2026, 4 de septiembre). Meta minimizes role of token maxing in employee evaluations. *InfoWorld / CIO*. https://www.cio.com/article/4218661/meta-minimizes-role-of-token-maxing-in-employee-evaluations-2.html (Cita un reporte de The Information.)

Lazzaro, S. (2025, 18 de junio). AI coding trackers are here. Proceed with caution. *LeadDev*. https://leaddev.com/reporting/ai-coding-tool-trackers-proceed-with-caution

Foucault, M. *Vigilar y castigar: nacimiento de la prisión*.

Foucault, M. *Microfísica del poder*.

Han, B.-C. *Psicopolítica*.

Han, B.-C. *La sociedad del cansancio*.

Dekker, S. *Just Culture: Balancing Safety and Accountability*. (Ashgate Publishing, 2007)

Allspaw, J. (2012). Blameless PostMortems and a Just Culture. Etsy, Code as Craft. (https://www.etsy.com/pl/codeascraft/blameless-postmortems)

Edmondson, A. (1999). Psychological safety and learning behavior in work teams. *Administrative Science Quarterly*.

Kerth, N. (2001). *Project Retrospectives: A Handbook for Team Reviews*. (Directiva Primaria.)

DORA / *Accelerate* (Forsgren, Humble y Kim).

Schwaber, K. y Sutherland, J. *La Guía de Scrum* (2020).

Anexos

- Anexo A. Plantilla de post-mortem sin culpa
- Anexo B. Pautas para dailys sin vigilancia